



no proceed

Continuing from the `title` construction onward (middle third of the file), with clear but not over-detailed explanations.

15. Build PR title and body

```
const title = isFinalMergeToDevelop
  ? `[Cascade Final] ${current.branch} -> ${nextBranch}`
  : `[Cascade] ${current.branch} -> ${nextBranch}`;
```

- If `isFinalMergeToDevelop` is true (last release -> develop):
 - Title: `[Cascade Final] release/app_2.00.00 -> develop.`
- Otherwise (release -> next release):
 - Title: `[Cascade] release/app_1.02.00 -> release/app_2.00.00.`

Gives a quick visual cue which PRs are final cascades.

```
const body = [
  `This PR was created automatically after PR #${mergedPrNumber} was merged into \`,
  ``,
  `### Cascade behavior`,
  isFinalMergeToDevelop
    ? `- This is the final cascade PR from the last eligible release branch into \`${current.branch}\`
      : `- Only the next downstream cascade PR is created.`,
  `- If this PR has conflicts, the cascade stops here.`,
  `- If this PR is immediately mergeable, it is merged automatically.`,
  `- If required branch conditions block merge, the PR remains open for review.`,
  isFinalMergeToDevelop
    ? `- Once this PR is merged, the release cascade is complete.`
    : `- Once this PR is merged, the next downstream cascade PR can be created autom
  ``,
  `### Details`,
  `- App: \`${current.app}\``,
  `- Current version: \`${current.version}\``,
  `- Source branch: \`${current.branch}\``,
  `- Target branch: \`${nextBranch}\``,
].join('\n');
```

Body structure:

- Intro: explains which PR triggered this cascade (`mergedPrNumber`) and which branch it merged into (`current.branch`).
- “### Cascade behavior”:
 - If final:
 - Says this is the last step into `nextBranch` (likely `develop`).
 - Otherwise:
 - Says only the `next` downstream cascade PR is created.
 - Then bullets describing:
 - Conflicts stop the cascade.
 - Clean PRs auto-merge.
 - Blocked PRs remain open.
 - Final vs non-final behavior text (cascade complete vs next step can be created).
- “### Details”:
 - App, current version, source branch, target branch, all in inline code for readability.

So every cascade PR contains clear documentation of why it exists and how the cascade behaves for that step.^[1]

16. Create the cascade PR

```
let createdPr;
try {
  createdPr = await github.rest.pulls.create({
    owner,
    repo,
    head: current.branch,
    base: nextBranch,
    title,
    body,
    maintainer_can_modify: false
  });
} catch (error) {
  if (error.status === 422) {
    core.warning(`Cascade PR was not created. This may mean there is no diff or an e
    return;
  }
  throw error;
}
```

- Calls GitHub’s “Create a pull request” API with:
 - `head: current.branch` (source release branch).
 - `base: nextBranch` (target release or `develop`).
 - `title, body` as built above.

- `maintainer_can_modify`: `false` so maintainers don't directly edit the source release branch; you resolve conflicts via separate feature branches.^[2] ^[1]
- On success, `createdPr` is the API response.
- Error handling:
 - If status 422 (Unprocessable Entity):
 - Often means “no diff” between branches or similar duplicate PR situation.
 - Logs a warning and exits gracefully.
 - Any other error is rethrown (workflow fails visibly).

```
const prNumber = createdPr.data.number;
core.info(`Created cascade PR #${prNumber}`);
```

- Extracts the new PR number (e.g. 51) and logs `Created cascade PR #51`.

17. Label the PR and poll mergeability

```
await addLabel(prNumber, isFinalMergeToDevelop ? 'cascade-final-pr' : 'cascade-pr')
```

- Adds label:
 - `cascade-final-pr` for final step into `develop`.
 - `cascade-pr` for intermediate release-to-release steps.
- Lets you filter PRs in GitHub UI and see the chain.^[1]

```
let prDetails = await pollMergeability(prNumber, 10, 2000);

core.info(
  `Final mergeability for PR #${prNumber}: mergeable=${prDetails?.mergeable}, mergeable_state=${prDetails?.mergeable_state}
`);
```

- Calls `pollMergeability`:
 - Up to 10 attempts, 2 seconds between.
 - Stops once `mergeable` is no longer `null`.^[3] ^[1]
- `prDetails` now includes:
 - `mergeable` (`true` / `false` / `null`)
 - `mergeable_state` (`clean`, `dirty`, `blocked`, `unstable`, `unknown`, etc.)
- Logs a line summarizing the final state after polling.

18. Case 1: mergeability unknown/pending

```
if (prDetails?.mergeable === null || prDetails?.mergeable_state === 'unknown') {
  await addLabel(prNumber, 'cascade-pending');
  await addComment(
    prNumber,
    [
      `The workflow could not determine mergeability for this cascade PR within the`,
      ``,
      `The PR has been left open for manual review or a later retry.`,
      `Observed state: mergeable=\`${prDetails?.mergeable}\`, mergeable_state=\`${prDetails?.mergeable_state}\``,
    ].join('\n')
  );
  core.info(`PR #${prNumber} remained unresolved after polling. Leaving it open.`);
  return;
}
```

- Condition: GitHub did not settle on mergeability:
 - `mergeable` is still `null`, or
 - `mergeable_state` is `"unknown"`.^[3]
- Action:
 - Label `cascade-pending`.
 - Add a comment explaining:
 - It couldn't determine mergeability within the 10 polls.
 - PR is left open for manual review or later retry.
 - Shows the observed state values.
 - Logs and returns.
- No auto-merge attempts; humans must decide when/if to re-run or manually merge.

19. Case 2: conflicts / not mergeable (dirty)

```
if (prDetails?.mergeable === false || prDetails?.mergeable_state === 'dirty') {
  await addLabel(prNumber, 'cascade-conflict');
  await addComment(
    prNumber,
    [
      `This cascade PR currently has merge conflicts or is not mergeable.`,
      ``,
      `isFinalMergeToDevelop`,
      `? `The final merge into \`${nextBranch}\` is paused until this PR is resolved`,
      `: `The cascade stops here until this PR is resolved and merged.`
    ].join('\n')
  );
  core.info(`PR #${prNumber} is not mergeable. Cascade stops here.`);
  return;
}
```

- Condition: GitHub reports:
 - `mergeable == false` OR `mergeable_state == 'dirty'`.
 - In practice, this means conflicts or non-mergeable state.^[3]
- Action:
 - Label `cascade-conflict`.
 - Comment:
 - Says the cascade PR has conflicts or is not mergeable.
 - If final step:
 - “The final merge into `nextBranch` is paused until this PR is resolved and merged.”
 - Else:
 - “The cascade stops here until this PR is resolved and merged.”
 - Logs and returns.

You then follow your standard pattern: fix conflicts via feature branch from the target release, merge that, and then this cascade PR becomes clean.

20. Case 3: clean → auto-merge

```
if (prDetails?.mergeable == true && prDetails?.mergeable_state == 'clean') {
  await mergePullRequest(prNumber);
  core.info(`PR #${prNumber} was merged immediately because it is in clean state.`)
  return;
}
```

- Condition: `mergeable == true` and `mergeable_state == 'clean'`.^[3]
- Action:
 - Calls `mergePullRequest(prNumber)` to perform a normal merge.
 - Logs that it merged immediately because it was clean.
 - Returns.

This is the “happy path” for both intermediate and final cascade steps.

21. Case 4: blocked by rules / checks

```
if (prDetails?.mergeable == true && prDetails?.mergeable_state == 'blocked') {
  await addLabel(prNumber, 'cascade-blocked');
  await addComment(
    prNumber,
    [
      `This cascade PR is currently blocked by required merge conditions or repository rules.`,
      `The workflow will not force-merge this PR.`,
    ]
  );
}
```

```

    `Observed state: mergeable=\`${prDetails?.mergeable}\`, mergeable_state=\`${prDetails?.mergeable_state}\`
    ].join('\n')
  );
  core.info(`PR #${prNumber} is blocked. Leaving it open for required conditions to be satisfied.`);
  return;
}

```

- Condition: `mergeable === true` but `mergeable_state === 'blocked'`.
 - Typically means required checks or approvals are not satisfied.^[4] ^[3]
- Action:
 - Label `cascade-blocked`.
 - Comment explains:
 - It's blocked by required merge conditions or rules.
 - The workflow will not force-merge.
 - Shows the observed state.
 - Logs and returns.

You or your team must satisfy the conditions (checks, reviews, etc.) and then merge manually. The automation intentionally respects branch protections.

22. Case 5: unstable, with extended retries

```

if (prDetails?.mergeable === true && prDetails?.mergeable_state === 'unstable') {
  const unstableMaxAttempts = 12;
  const unstableDelayMs = 10000;

  core.info(`PR #${prNumber} is in unstable state. Retrying for up to ${unstableMaxAttempts} attempts.`);

  for (let attempt = 1; attempt <= unstableMaxAttempts; attempt++) {
    await sleep(unstableDelayMs);
    prDetails = await getPullRequest(prNumber);

    core.info(
      `Unstable retry ${attempt}/${unstableMaxAttempts} for PR #${prNumber}: mergeable=${prDetails?.mergeable}, mergeable_state=${prDetails?.mergeable_state}`
    );
  }
}

```

- Initial condition: `mergeable === true` and `mergeable_state === 'unstable'`.
 - GitHub is still computing checks or something else is in flux.^[3]
- Sets:
 - `unstableMaxAttempts = 12`
 - `unstableDelayMs = 10000` (10 seconds)
- Logs that it will retry up to 12 times.
- Loop:
 - Sleeps 10 seconds.

- Fetches fresh PR data via `getPullRequest`.
- Logs each retry's `mergeable` and `mergeable_state`.

Inside the loop, three sub-cases:

22a. Becomes clean during retries

```
if (prDetails?.mergeable === true && prDetails?.mergeable_state === 'clean') {
  await mergePullRequest(prNumber);
  core.info(`PR #${prNumber} was merged after unstable state resolved to clean.`);
  return;
}
```

- If during retries it hits `true` / `clean`:
 - Auto-merge and log success, then return.

22b. Becomes dirty (non-mergeable) during retries

```
if (prDetails?.mergeable === false || prDetails?.mergeable_state === 'dirty') {
  await addLabel(prNumber, 'cascade-conflict');
  await addComment(
    prNumber,
    [
      `This cascade PR became non-mergeable while waiting for unstable state to resolve`,
      ``,
      `isFinalMergeToDevelop`,
      `? 'The final merge into \`${nextBranch}\` is paused until this PR is resolved'`,
      `: 'The cascade stops here until this PR is resolved and merged.'`,
      `Observed state: mergeable=\`${prDetails?.mergeable}\`, mergeable_state=\`${prDetails?.mergeable_state}\``
    ].join('\n')
  );
  core.info(`PR #${prNumber} became non-mergeable during unstable retry window.`);
  return;
}
```

- If it flips to `mergeable=false` OR state `dirty`:
 - Label `cascade-conflict`.
 - Comment that it became non-mergeable while waiting, with final vs non-final wording, and the observed state.
 - Log and return.

22c. Becomes blocked during retries

```
if (prDetails?.mergeable === true && prDetails?.mergeable_state === 'blocked') {
  await addLabel(prNumber, 'cascade-blocked');
  await addComment(
    prNumber,
    [
      `This cascade PR became blocked by required merge conditions or repository`
    ]
  );
  return;
}
```

```

        ```,
 `The workflow will not force-merge this PR.`,
 `Observed state: mergeable=\`${prDetails?.mergeable}\`, mergeable_state=\`${prDetails?.mergeable_state}\`
].join('\n')
);
 core.info(`PR #${prNumber} became blocked during unstable retry window.`);
 return;
}
}

```

- If it becomes `true` / blocked during retries:
  - Label `cascade-blocked`.
  - Comment that it became blocked by required conditions while waiting; workflow won't force-merge.
  - Log and return.

If the loop finishes all 12 attempts without any of those returns:

```

await addLabel(prNumber, 'cascade-unstable');
await addComment(
 prNumber,
 [
 `This cascade PR remained in an unstable state and was not merged automatically`,
        ```,
        `The workflow retried for a longer window, but the PR did not become clean.`,
        `Observed state: mergeable=\`${prDetails?.mergeable}\`, mergeable_state=\`${prDetails?.mergeable_state}\`
    ].join('\n')
);
core.info(`PR #${prNumber} remained unstable after extended retry window. Leaving`);
return;
}

```

- Label `cascade-unstable`.
- Comment that:
 - It stayed unstable and did not become clean during extended retries.
 - Observed state is included.
- Logs and returns.

This extended handling gives GitHub more time while still eventually backing off and marking the PR clearly.

23. Final fallback: manual review

```

await addLabel(prNumber, 'cascade-manual-review');
await addComment(
    prNumber,
    [
        `This cascade PR reached an unhandled mergeability state and requires manual review`
    ]
);

```



```

    ``',
    `Observed state: mergeable=\`${prDetails?.mergeable}}`, mergeable_state=\`${pr
    ].join('\n')
  );
  core.warning(`PR #${prNumber} reached an unhandled state. Leaving it open.`);

```

- This is reached only if none of the previous conditions matched (a future or unexpected combination).
- Adds label `cascade-manual-review`.
- Comments that the PR hit an unhandled mergeability state and needs manual review, with the observed state.
- Logs a **warning** that it reached an unhandled state.

At this point, the script ends; everything else is left for humans.

You now have:

- First answer: up through building `existingPr`.
- This answer: title/body → PR creation → labels → mergeability handling → final fallback.

Is there any particular branch of the mergeability logic (pending, conflict, clean, blocked, unstable, manual-review) that you'd like a compact "timeline example" for (e.g., step-by-step for one real merge scenario), or does this level of explanation already cover what you need to confidently release it?



1. `cascade-next-pr.yml`
2. <https://docs.github.com/en/repositories/configuring-branches-and-merges-in-your-repository/managing-rulesets/available-rules-for-rulesets>
3. <https://docs.github.com/en/enterprise-server@3.14/repositories/configuring-branches-and-merges-in-your-repository/managing-rulesets/available-rules-for-rulesets>
4. <https://docs.github.com/en/repositories/configuring-branches-and-merges-in-your-repository/managing-protected-branches/about-protected-branches>